Dynamic Programming Approach

It divides the problem into series of overlapping sub-problems.

Two features

1) Optimal Substructure
2) Overlapping Subproblems. – Fibonacci Series $f(n) = f(n-1) + f(n-2)$

Dynamic programming is an algorithmic technique with the following properties

- It is mainly an optimization over plain recursion wherever we see a recursive solution that has repeated calls for the same inputs, we can optimize it using Dynamic Programming.
- The idea is to simply store the results of subproblems so that we do not have to re-compute them when needed later. This simple optimization typically reduces time complexities from exponential to polynomial.

↳ Like divide and conquer method, DP solves problems by combining the solution of subproblems. Moreover, DP algorithm solves each sub-problem just once and then saves its answers in a table, thereby avoiding the work of re-computating the answer every time. Two main properties of a problem suggest that the given problem can be solved using Dynamic programming.

(1) Overlapping sub-Problems

Similar to Divide-and-Conquer approach, Dynamic Programming also combines solutions to sub-problems. It is mainly used where the solution of one sub-problem is needed repeatedly. The computed solutions are stored in table, so that these don't have to be re-computed. Hence, this technique is needed where overlapping sub-problem exists.

For ex – Fibonacci numbers have many overlapping subproblem.

(2) Optimal Sub-Structure :-

A given problem has optimal substructure property if the optimal solution of the given problem can be obtained using optimal solutions of its sub-problems.

For Ex- Shortest path problem has the following optimal substructure property -

If a node x lies in the shortest path from a source node u to destination v ($u \to v$), then the shortest path from u to v is the combination of shortest path from $u \to x$ & the shortest path from $x \to v$

The standard All pair shortest path algorithms like Floyd-Warshall and Bellman-Ford are the examples of Dynamic programming

Steps of Dynamic Approach :-

DP algorithm is designed using following 4 steps

1. Characterize the structure of an optimal solutions
2. Recursively define the value of an optimal solutions
3. Compute the value of an optimal solution, typically in a bottom-up fashion
4. Construct an optimal solution from the computed information.

Difference between Dynamic Programming & Divide and Conquer Technique

Divide and Conquer algorithm partition the problem into disjoint sub problem recursively and then combine their solution to solve the original problems. Divide & Conquer algorithm can be a Bottom-up-approach and Top down approach.

Dynamic Programming is used when the subproblems are not independent, e.g. when they share the same problems multiple times. Dynamic Programming solves each sub problem just once and stores the

result in a table so that it can be repeatedly retrieved if needed again.

Dynamic programming is a Bottom-up approach we solve all possible small problems and then combine to obtain solutions for bigger problems.

Dynamic programming is a paradigm of algorithm design in which an optimization problem is solved by combinang tion of achieving sub-problem solutions and appeaning to the "principle of optimality".

Divide & conquer Method	Dynamic Programming
(1) It deals (involves) 3 steps at each level of recursion: Divide the problem into a number of sub problem. Conquer the sub problems by solving them recursively. Combine the solution to the sub problems into the solution for orignal sub problems.	(1) It involves the sequence of 4 steps: ↳ Characterize the structure of optimal solutions ↳ Recursively define the values of optimal solu^ns ↳ Compute the value of optimal solu^ns in Bottom-up minimum. ↳ Construct an optimal solutions from computed infor^n.
(2) It is Recursive	(2) It is not Recursive
(3) It does more work on sub problems and hence has more time consumption.	(3) It solves sub problems only once and then stores in the table.
(4) It is a top-down approach	(4) It is a Bottom-up approach
(5) In this sub problems are independent of each other.	(5) In this sub problems are interdependent
(6) For ex - Merge sort & Binary Search etc.	(6) For ex - Matrix Multiplication.

Elements of Dynamic Programming:

There are basically 3 elements that characterize a dynamic programming algorithm.

1. Substructure:

   Decompose the given problem into smaller sub problems. Express the solution of the original problem in terms of the solution for smaller problems.

2. Table structure:

   After solving the sub-problems, store the results to the sub problems in a table. This is done because sub problem solutions are reused many times, and we do not want to repeatedly solve the same problem over and over again.

3. Bottom-up Computation:

   Using table, combine the solution of smaller sub problems to solve larger sub problems & eventually arrives at a solution to complete problem.

Single Source shortest path (SSSP) :-

SSSP problem finds the shortest path from a single vertex to all other vertices in a graph. The all-pairs shortest path (APSP) problem finds the shortest path between every pair of vertices in a graph.

Example can be asked → Find the shortest path from a source vertex to all other vertices in the graph.

Algorithms like- Dijkstra's (Greedy approach) & Bellman Ford (Dynamic programming Approach) solve this problem.

All pairs shortest path (APSP)

Find the shortest path between every pair of vertices in the graph.

Algorithms like Floyd - Warshall can solve the problem.

Floyd - Warshall Algorithm :-

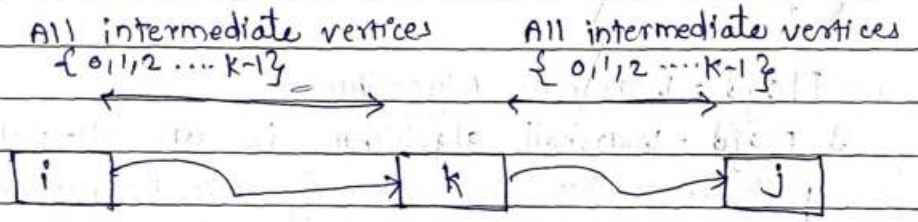
- Floyd - Warshall algorithm is an all-pair shortest path algorithm that uses Dynamic Programming to find the shortest distances between every pair of vertices in a graph; unlike Dijkstra's, and Bellman - Ford which are single source shortest path algorithm.

This algorithm works for both the directed and undirected weighted graphs and can handle graphs with both positive and negative weight edges. But does not work for the graphs with negative cycles (where the sum of the edges in a cycle is negative)

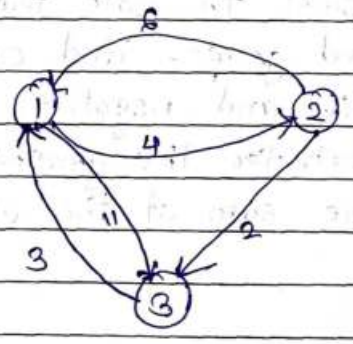
Idea Behind Floyd Warshall Algorithm:

Suppose: we have a graph with V vertices from 0→V-1
Now we have to evaluate a dist[ ][ ] where dist[i][j] represents the shortest path between vertex i to j
Let us assume that vertices i to j have intermediate nodes. The idea behind Floyd Warshall algo. is to treat each and every vertex k from 0 to V-1 as an intermediate node one by one. When we consider the vertex k, we must have considered vertices from 0 to k-1 already, so we use the shortest paths built by previous vertices to build shorter paths with vertex k included.

The following figure shows the above optimal substructure property in Floyd Warshall algorithm:



Solve the all pairs shortest path problem for the given graph.



→ Optimal substructure formula for Floyd's algorithm,

$$D^k[i,j] = \min\{D^{k-1}[i,j], D^{k-1}[k,j]\}$$

$$D^0 = \begin{array}{c|ccc} & 1 & 2 & 3 \\ \hline 1 & 0 & 4 & 11 \\ 2 & 6 & 0 & 2 \\ 3 & 3 & \infty & 0 \end{array}$$

Distance [1,1], [2,2] & [3,3] is zero.
There is no direct path from [3,2] therefore it is ∞

Iteration 1 (K=1) (Distance of all the vertices via 1)

$D^1[1,2] = \min\{D^0[1,2], D^0[1,1] + D^0[1,2]\}$
$= \min\{4, 0+4\} = 4$

$D^1[1,3] = \min\{D^0[1,3], D^0[1,1] + [1,3]\}$
$= \min\{11, 0+11\} = 11$

$D^1[2,1] = \min\{D^0[2,1], D^0[2,1] + [1,1]\}$
$= \min\{6, 6+0\} = 6$

$D^1[2,3] = \min\{D^0[2,1], D^0[2,1] + [1,3]\}$
$= \min\{2, 6+11\}$
$= 2$

$D^1[3,1] = \min\{D^0[3,1], D^0[3,1] + D^0[1,1]\}$
$= \min\{3, 3+0\} = 3$
$= 3$

$D^1[3,2] = \min\{D^0[3,2], D^0[3,1] + D^0[1,2]\}$
$= \min\{\infty, 3+4\}$
$= \min\{\infty, 7\}$
$= 7$

	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

D^1

	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

Iteration 2 : (K = 2)

D^2

	1	2	3
1	0	4	
2	6	0	2
3		7	0

$D^2\{1,3\} = \min\{D^1[1,3], D^1[1,2] + D^1[2,3]\}$

$= \min\{11, 4+2\}$

$= 6$

$D^2\{3,1\} = \min\{D^1[3,1], D^1[3,2] + D^1[2,1]\}$

$\min\{3, 7+6\}$

$= 3$

D^2

	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

D^3

	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0

with updated values

Iteration 3 : (K = 3)

D^3

	1	2	3
1	0		6
2		0	2
2	3	7	0

$D^3\{1,2\}$ & $D^3\{2,1\}$

$D^3\{1,2\} = \min\{D^2\{1,2\}, D^2\{1,3\} + D^2\{3,2\}\}$

$= \min\{4, 6+7\}$

$= 4$

$D^3\{2,1\} = \min\{D^2\{2,1\}, D^2\{2,3\} + D^2\{3,1\}\}$

$= \min\{6, 2+3\} = 5$

Bellman-Ford (BF) algorithm.

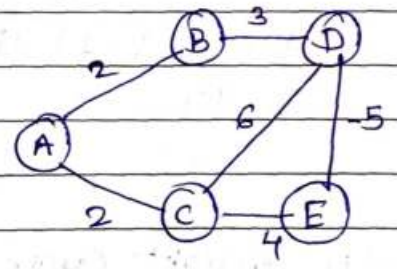
The Bellman-Ford algorithm is a single-Path Shortest Path (SSSP) algorithm. This means it can find the shortest path from one node to any other node. However, BF is not ideal for most SSSP problem because it has a time complexity of $O(EV)$ It is better to use Dijkstra's Algorithm which is much faster. It is on the order of $\Theta((E+V)\log(V))$ when using a binary heap priority queue.

However, Dijkstra's algorithm can fail when the graph has negative cycles and determine where they occur.

Finding negative cycles can be useful in many types of applications. One particularly neat application arises in finance when performing an arbitrage between two or more markets.

This algo. begins with a starting vertex and calculates the distances between other vertices that a single edge can reach. It then searches for a path with two edges, and so on.

The Bellman-Ford algo. uses the Bottom-up approach



	B	C	D	E
0	∞	∞	∞	∞
0	2	2	∞	∞
0	2	2	2	6
0	2	2	3	6
0	2	2	3	6

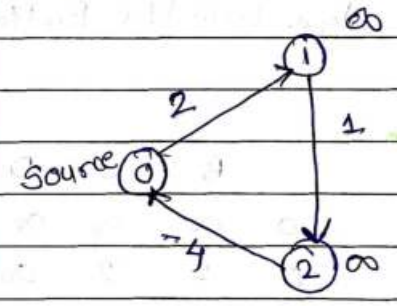
Based on Principal of Relaxation

Given a weighted directed graph $G=(V,E)$, and a weight function w, the Bellman-Ford algorithm returns a Boolean value indicating whether a negative weight cycle can be obtained from the source.

If such a cycle exists, the algorithm computes the shortest paths and their weights. The algorithm returns TRUE if a graph has no negative weight cycles that can be reached from the source.

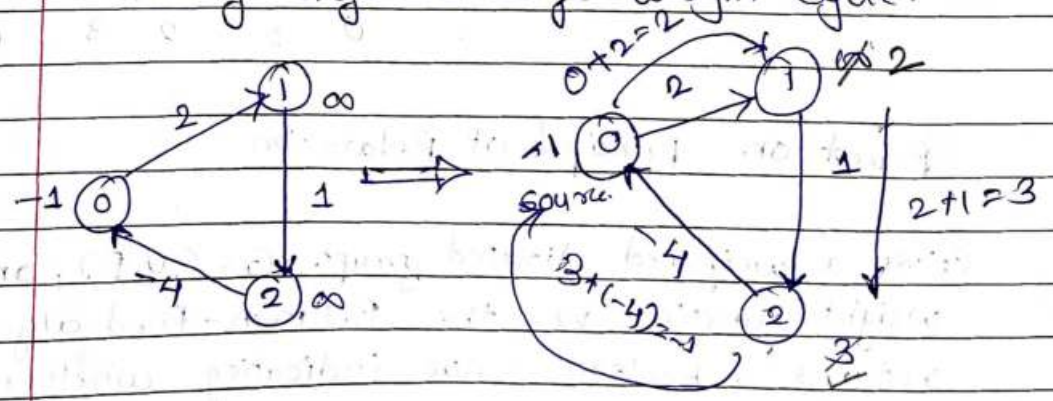
Algorithm

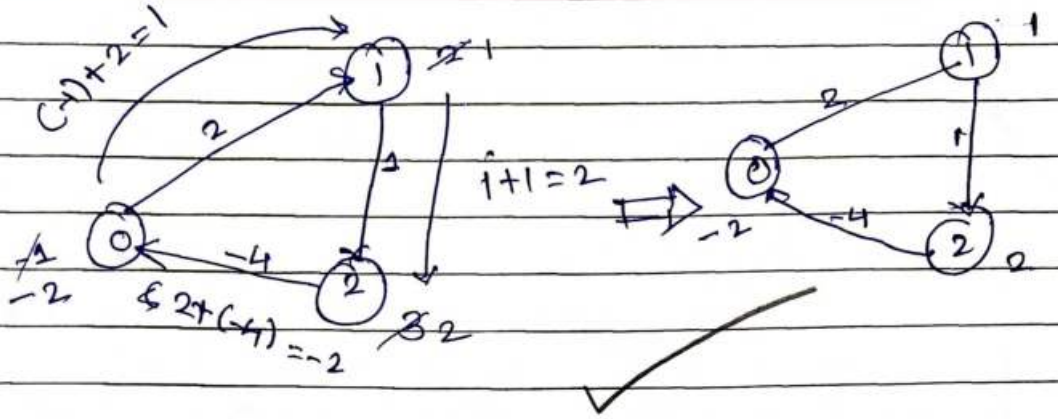
1. Except the source node, set the distance values of all nodes to infinity (∞)
2. Repeat ($V-1$) times // where $V \rightarrow$ No. of vertices
   if distance [u] + cost (u, v) < distance (u)
   // where $u \rightarrow$ source node
   // $v \rightarrow$ destination node
   then update distance [u]
   else
   skip.
3. All vertices should be relaxed once more. We have a negative edge weight cycle if we find a new shortest value; otherwise, we do not have a negative edge weight cycle and can return the answer

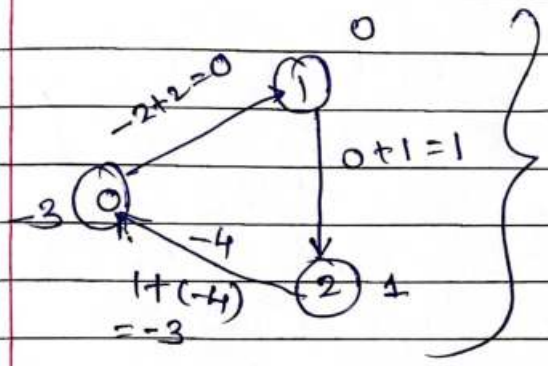


Suppose graph has 3 vertices, we will relax it for ($V-1$) time which is 2 times.

Finding negative edge weight cycle.

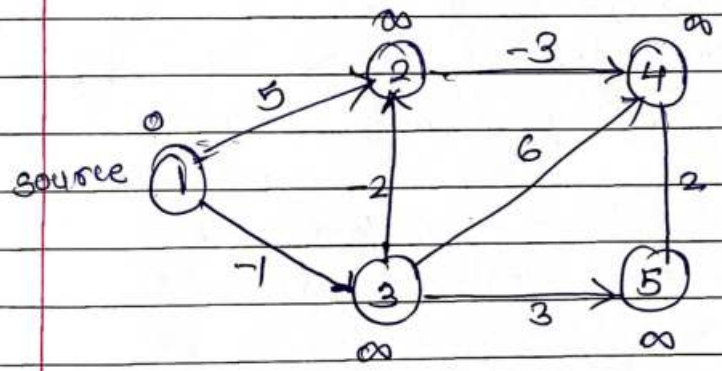






Third iteration

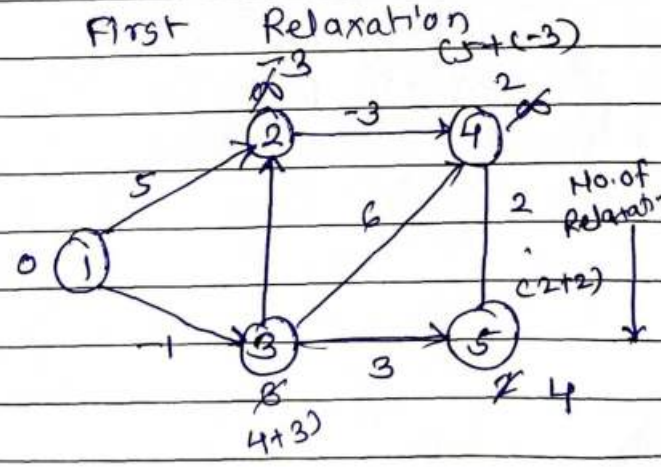
we must relax twice, but after 3rd time, we should find any shortest path but after the third time, we have the smallest costas 0. which should not happen, indicating that there is a negative weight edge cycle present in the graph.



List of edges

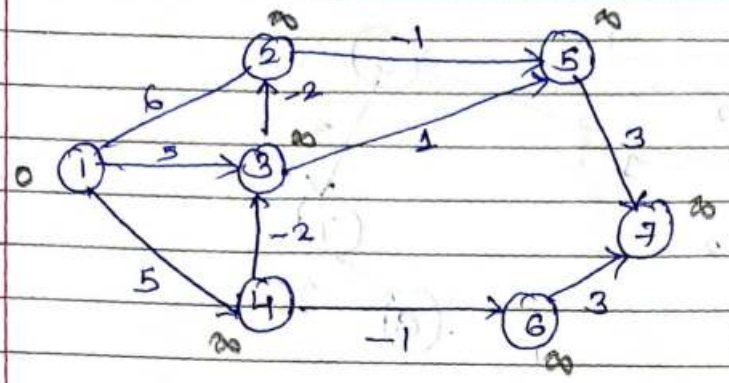
1. (1,2)
2. (1,3)
3. (2,4)
4. (3,2)
5. (3,4)
6. (3,5)
7. (4,5)

First Relaxation



No. of Relaxation	vertices → 1	2	3	4	5	6	7
1	0	-3	-1	2	2		
2							
3							
4							
5							

Bellman-Ford Example.

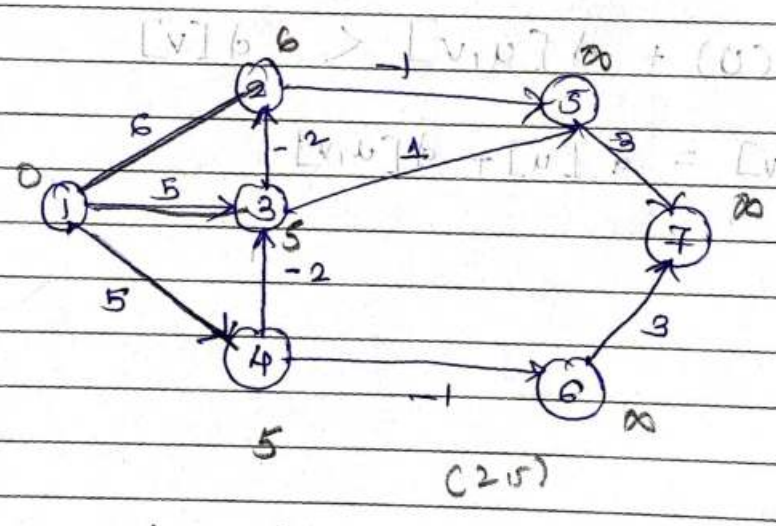


V = 7

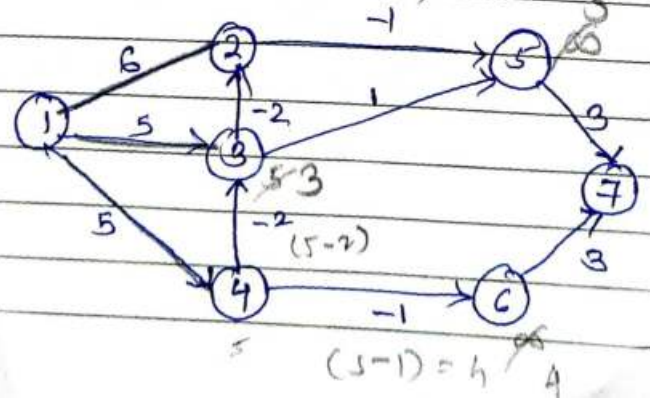
Relaxation = 7-1 = 6

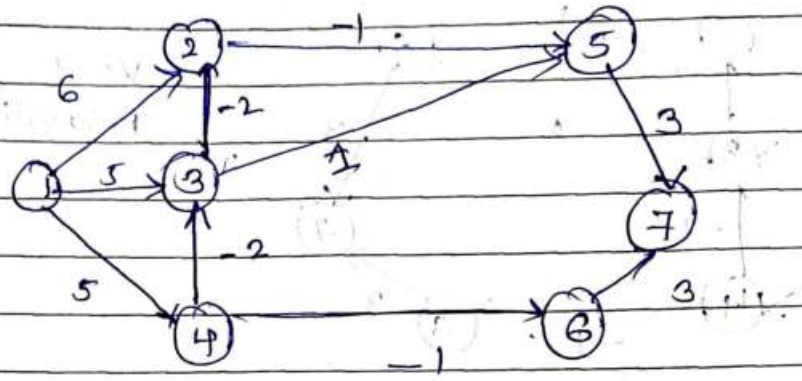
(1,2) (1,3) (1,4) (2,5) (3,2) (3,5) (4,3) (4,6) (5,7) (6,7)

	1	2	3	4	5	6	7
0	∞	∞	∞	∞	∞	∞	∞
1	0	6	5	5	∞	∞	∞
2							
3							
4							



	1	2	3	4	5	6	7
0							
1							
2	0	3	(6-1) 5	5	5	4	∞





0	1	2	3	4	5	6	7
0	∞	∞	∞	∞	∞	∞	∞
1	0	6	5	5	∞	∞	∞
2	0	3	3	5	5	4	∞
3	0	1	3	5	2	4	5
4	0	1	3	5	0	4	3
5	0	1	3	5	0	4	3
6	0	1	3	5	0	4	3

$$d(U) + d[u,v] < d[v]$$

Then change

$$d[v] = d[u] + d[u,v]$$

0/1 Knapsack Problem

weights = {3, 4, 6, 5} $\boxed{w=8}$

Profits = {2, 3, 1, 4} $\boxed{n=4}$

capacity →

p_i	w_i	i ↓ \ $\boxed{w}$ →	0	1	2	3	4	5	6	7	8	
		0	0	0	0	0	0	0	0	0	0	
2	3	1	0	0	0	2 ←	2	2	2	2	2	← 1
3	4	2	0	0	0	2 ←	3	3	3	5	5	← 0
4	6	3	0	0	0	2	3	4	4	5	6	← 0
1	6	4	0	0	0	2	3	4	4	5	$\boxed{6}$	← 1

$$m[i,w] = \max(m[i-1,w],\ m[i-1,w-w[i]] + p[i])$$

weight is zero

$[2,4] = \max(3 + \overset{(4-4)}{0},\ 2)$ $[2,5] = \max(\cancel{4} + \overset{(5-4)\ 1}{0},\ 2)$

$= \max(3,2)$

$= \underline{3}$

$[2,6] = \max(3 + \overset{(6-4)=2}{0},\ 2)$ (weight at 2 is zero) previous value.

$= \underline{3}$

$[2,7] = \max(3 + \overset{(7-4)=3}{2},\ 2)$ (value at 3 is 2)

$= \max(5,2)$

$= 5$

$[2,8] = \max(3 + \overset{(8-4)=4}{2},\ 2)$ (value at 4 is 2)

$= 5$

$x_i = \{1, 0, 0, 1\}$

Item : 4 Max = 5

$\{w_1, w_2, w_3, w_4\} = \{2, 1, 3, 2\}$

$\{P_1, P_2, P_3, P_4\} = \{12, 10, 20, 15\}$

capacity →

P_i	w_i	i ↓ / w →	0	1	2	3	4	5	6	7	8	
		0	0	0	0	0	0	0	0	0	0	
10	1	1	0	10	10	10	10	10	10	10	10	← 1
12	2	2	0	0	12	22	22	22	22	22	22	← 1
20	2	3	0	0	20	22	22	32	42	42	42	← 0
20	3	4	0	0	20	22	22	40	42	42	42	← 1

max(12+10, 10) = 22 max(12+10, 18) = 22

max(20+0, 12) max(20+0, 22)

[3,4] = (20+0, 22)

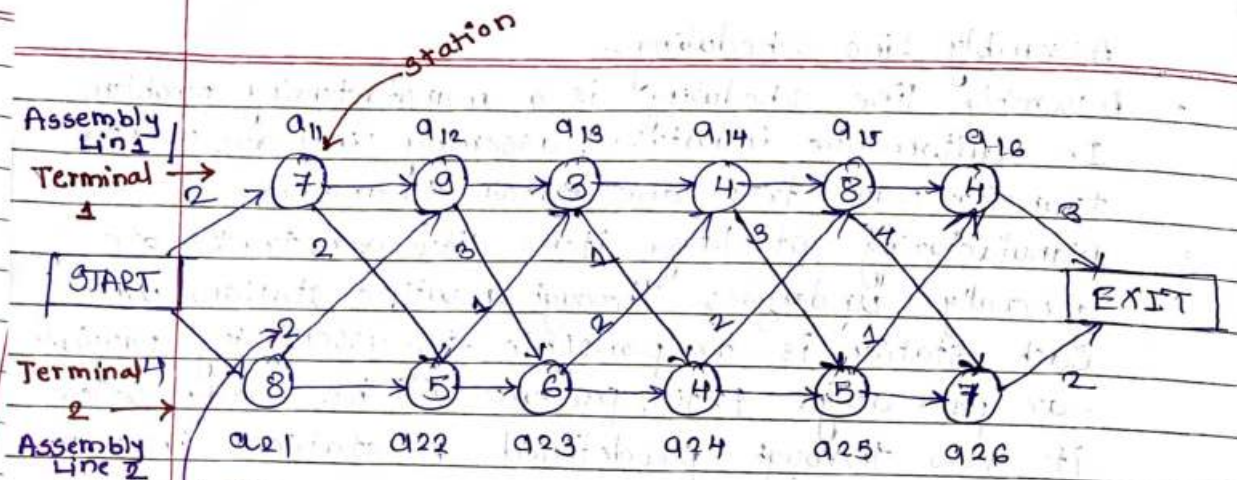
[3,5] = (20+12, 22) = 32, 22 = 32 [3,6] = (20+22, 22) = 42, 22

[3,7] = (20+22, 22) = 42 [3,8] = (20+22, 22) = 42

[4,6] = (20+20, 42) = 42 [4,7] = (20+22, 42) 42

Assembly Line Scheduling -

- Assembly line scheduling is a manufacturing problem. In automobile industries, assembly lines are used to transfer parts from one station to another.
- Manufacturing of large items like car, trucks etc generally undergoes through multiple stations, where each station is responsible for assembling particular part only. Entire product will be ready after it goes through predefined n stations in sequence.
- For ex - manufacturing of car may be done in several stages like engine fitting, colouring, light fitting, fixing of controlling system, gates, seats and many other things.
- The particular task is carried out at the station dedicated to that task only. Based on requirement, there may be more than one assembly line.
- In case of two assembly lines, if the load at station j of assembly line 1 is very high, then the components are transferred to station j of assembly line 2, the converse is also true. This helps to speed up the manufacturing process.



$e = \{2, 4\}$ $x = \{3, 2\}$

P_{ij}	1	2	3	4	5	6
1	7	9	3	4	8	4
2	8	5	6	4	5	7

t_{ij}	1	2	3	4	5
1	2	3	1	3	4
2	2	1	2	2	1

Step 1.

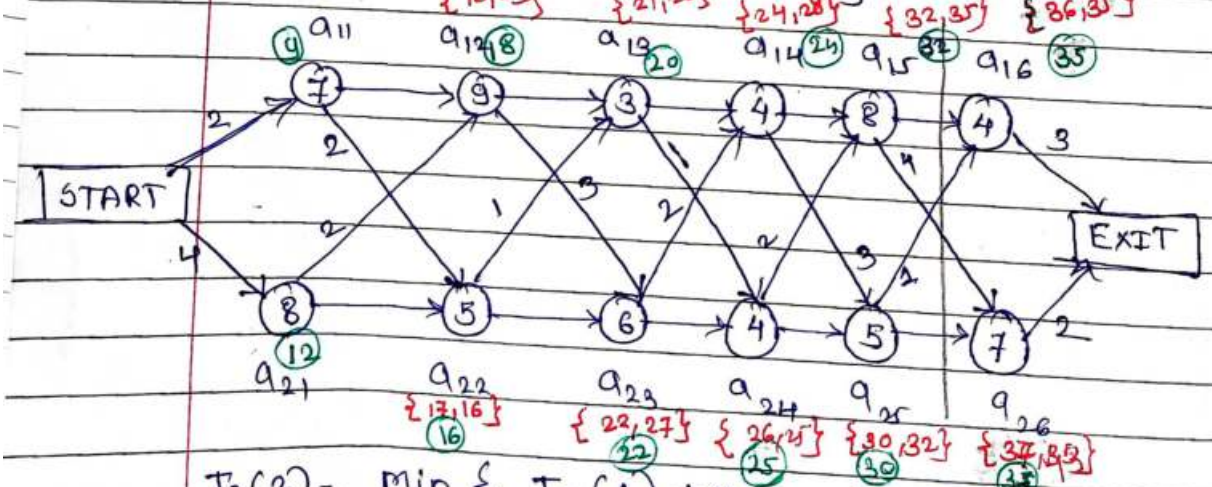
$T_1(1) = e_1 + a_{11} = 2 + 7 = 9$ Terminal 1 - station 1

$T_2(1) = e_2 + a_{21} = 4 + 8 = 12$ Terminal 2 - Statation 1

Step 2

$T_1(2) = \min\{T_1(1) + a_{12}, T_2(1) + t + a_{12}\}$ ← transportation cost

$= \min\{9+9, 12+2+9\} = 18$



$T_2(2) = \min\{T_2(1) + a_{22}, T_1(1) + t + a_{22}\}$

$= \min\{\underset{17}{12+5}, \underset{16}{9+2+5}\} = 16$

$T_1(3) = \min\{T_1(2) + a_{13}, T_2(2) + t + a_{13}\}$

$= \min\{21, \underset{20}{16+1+3}\} = 20$

$T_2(3) = \text{Min}\{T_2(2) + a_{23},\ T_1(2) + t + a_{23}\}$

$= \text{Min}\{16 + 6,\ 18 + 3 + 6\}$

$= \text{Min}\{22,\ 27\}$

$T_2(3) = 22$

$T_1(4) = \text{Min}\{T_1(3) + a_{14},\ T_2(3) + t + a_{14}\}$

$= \text{Min}\{20 + 4,\ 22 + 2 + 4\} = 24$

$T_2(4) = \text{Min}\{T_2(3) + a_{24},\ T_1(3) + t + a_{24}\}$

$= \text{Min}\{\underset{26}{22 + 4},\ \underset{25}{20 + 1 + 4}\}$

$= 25$

$T_1(5) = \text{Min}\{T_1(4) + a_{15},\ T_2(4) + t + a_{15}\}$

$= \text{Min}\{24 + 8,\ 25 + 2 + 8\}$

$= \text{Min}\{32,\ 35\}$

$= 32$

$T_2(5) = \text{Min}\{T_2(4) + a_{25},\ T_1(4) + t + a_{25}\}$

$= \text{Min}\{25 + 5,\ 24 + 3 + 5\}$

$= \text{Min}\{30, 32\}$

$= 30$

$T_1(6) = \text{Min}\{T_1(5) + a_{16},\ T_2(5) + t + a_{16}\}$

$= \text{Min}\{32 + 4,\ 30 + 1 + 4\}$

$= \text{Min}\{36,\ 35\}$

$= 35$

$T_2(6) = \text{Min}\{T_2(5) + a_{26},\ T_1(5) + t + a_{26}\}$

$= \text{Min}\{30 + 7,\ 32 + 4 + 7\}$

$= \text{Min}\{37,\ 43\}$

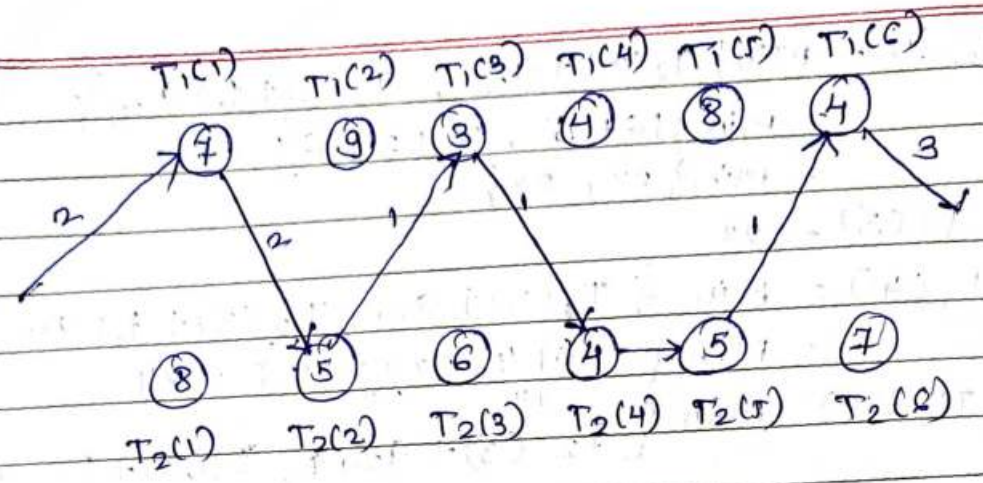
$T_2(6) = 37$

Step7 Optimal schedule : $\text{Min}(T_1(6) + \text{terminal cost of 1},\ T_2(6) + \text{Terminal cost of 2})$

$= \text{Min}\{35 + 3,\ 37 + 2\}$

$= \text{Min}\{38,\ 39\}$

$= 38$ (Assembly line 1)



$2+7+2+5+1+3+4+5+1+3 = 38$

Apply Floyd's Method To find the shortest path for below mentioned all pairs.

$$\begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix}$$

→ Optimal substructure formula for Floyd's algorithm

$$D^K[i,j] = \min\{D^{K-1}[i,j],\ D^{K-1}[i,K] + D^{K-1}[K,j]\}$$

$$D^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{array}$$

Iteration 1 : $k = 1$

$$D^1 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & & \\ \infty & & 0 & \\ 6 & & & 0 \end{bmatrix} \end{array}$$

$$D^1[2,3] = \min\{D^0[2,3],\ D^0[2,1] + [1,3]\}$$
$$= \min\{\infty,\ 2+3\}$$
$$= 5$$

$$D[2,4] = \min\{D^0[2,4],\ D^0[2,1] + [1,4]\}$$
$$= \min\{\infty,\ 2+\infty\}$$
$$= \infty$$

$$D[3,2] = \min\{D^0[3,2],\ D^0[3,1] + D^0[1,2]\}$$
$$= \min\{7,\ \infty+\infty\} = 7$$

$$D[3,4] = \min\{D^0[3,4],\ D^0[3,1] + D^0[1,4]\}$$
$$= \min\{1,\ \infty+\infty\}$$
$$= 1$$

$$D^1[4,2] = \min\{D^0[4,2],\ D^0[4,1] + D^0[1,2]\}$$
$$= \{\infty,\ 6+\infty\}$$
$$= \infty$$

$$D^1[4,3] = \min\{D^0[4,3],\ D^0[4,1] + D^0[1,3]\}$$
$$= \{\infty,\ 6+3\}$$
$$= 9$$

$$D^1 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & \infty & 3 & \infty \\ 2 & 2 & 0 & 5 & \infty \\ 3 & \infty & 7 & 0 & 1 \\ 4 & 6 & \infty & 9 & 0 \end{array}$$

Iteration 2 : $K = 2$

$$D^2 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & \infty & & \\ 2 & 2 & 0 & 5 & \infty \\ 3 & & 7 & 0 & \\ 4 & & \infty & & 0 \end{array}$$

$$D^2[1,3] = \min\{D^1[1,3],\ D^1[1,2] + D^1[2,3]\}$$
$$= \{3,\ \infty+5\}$$
$$= 3$$

$$D^2[1,4] = \min\{D^1[1,4],\ D^1[1,2] + D^1[2,4]\}$$
$$= \{\infty,\ \infty+\infty\}$$
$$= \infty$$

$$D^2[3,1] = \min\{D^1[3,1],\ D^1[3,2] + D^1[2,1]\}$$
$$= \min\{\infty,\ 7+2\} = 9$$

$$D^2[3,4] = \min\{D^1[3,4],\ D^1[3,2] + D^1[2,4]\}$$
$$= \min\{1,\ 7+\infty\}$$
$$= 1$$

$D^2[4,1] = \min\{D^1[4,1],\ D^1[4,2] + D^1[2,1]\}$

$= \min\{6,\ \infty+2\} = 6$

$D^2[4,3] = \min\{D^1[4,3],\ D^1[4,2] + D^1[2,3]\}$

$= \min\{9,\ \infty+5\}$

$= 9$

$$D^2 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & \infty & 3 & \infty \\ 2 & 2 & 0 & 5 & \infty \\ 3 & 9 & 7 & 0 & 1 \\ 4 & 6 & \infty & 9 & 0 \end{array}$$

$k = 3$

$$D^3 = \begin{bmatrix} 0 & \infty & 3 & \\ 2 & & 5 & \\ 9 & 7 & 0 & 1 \\ 6 & & 9 & \end{bmatrix}$$